📄 Manual de Especificaciones de Requerimientos (SRS)

Proyecto: Sistema de Tesorería "Dark Riders"

Versión: 3.0 (Release PWA)

Desarrollado por: ToxiroDigital

Fecha: 23 de Enero, 2026

---

1. Introducción

1.1 Propósito

El propósito de este documento es definir los requerimientos funcionales y no funcionales para el sistema de gestión financiera del club de motociclistas "Dark Riders". El sistema tiene como objetivo digitalizar el flujo de caja, automatizar el control de morosidad y transparentar las finanzas hacia los socios.

1.2 Alcance

El sistema es una **Aplicación Web Progresiva (PWA)** que permite:

- Registro y categorización de ingresos y egresos.
- Gestión de base de datos de socios.
- Cálculo automático de deudas y cuotas mensuales.
- Visualización de estados financieros en tiempo real.
- Acceso diferenciado por roles (Directiva vs Socios).

---

2. Descripción General

2.1 Perfiles de Usuario (Actores)

- **Administrador / Tesorero:** Usuario con permisos totales. Puede registrar transacciones, crear socios, generar deudas masivas y visualizar reportes estratégicos.
- **Socio:** Usuario con permisos de lectura limitada. Puede visualizar su propio estado de cuenta, historial de pagos y los gráficos globales de la salud financiera del club.

2.2 Entorno Operativo

El sistema opera en un servidor VPS Linux gestionado por **ToxiroDigital**, utilizando contenedores Docker. Es accesible desde cualquier navegador web moderno (Chrome, Safari, Firefox) y es instalable en dispositivos móviles (Android/iOS).

---

3. Requerimientos Funcionales (RF)

Módulo 1: Autenticación y Seguridad

- **RF-01:** El sistema debe solicitar correo electrónico y contraseña para ingresar.
- **RF-02:** Las contraseñas deben almacenarse encriptadas utilizando el algoritmo **Bcrypt**.
- **RF-03:** El sistema debe bloquear el acceso a vistas administrativas si el usuario logueado tiene rol de "Socio".

Módulo 2: Gestión de Transacciones (Core)

- **RF-04 Registro de Movimientos:** El sistema debe permitir registrar Ingresos y Egresos con fecha, monto y descripción.
- **RF-05 Lógica de Ingresos (Árbol de Decisión):**
    - Si el ingreso proviene de un **Socio**, se debe seleccionar al socio de una lista.
    - Si es de Socio y el motivo es **"Cuota"**, el sistema debe desplegar las cuotas pendientes para marcarlas como pagadas.
    - Si es de Socio y el motivo es **"Otro"**, debe permitir ingresar una descripción manual (ej: Parches).
    - Si el ingreso es **Externo**, no se solicita socio y se pide descripción manual.
- **RF-06 Listado Histórico:** El sistema debe mostrar una tabla con los últimos 50 movimientos, permitiendo filtrar por Tipo (Ingreso/Egreso) y búsqueda por texto.

Módulo 3: Gestión de Morosidad y Cuotas

- **RF-07 Generación de Cuotas:** El sistema debe permitir al Administrador generar un cargo (deuda) masivo para un mes y año específico a todos los socios activos (excluyendo Administradores).
- **RF-08 Semáforo de Deuda:** El sistema debe calcular el porcentaje de cumplimiento de cada socio y asignar un estado visual (Verde: Al día, Naranja: Pendiente, Rojo: Crítico).
- **RF-09 Estado de Cuenta Personal:** El socio debe poder ver en su pantalla principal solo sus deudas pendientes y su historial de pagos.

Módulo 4: Dashboard e Inteligencia de Negocios

- **RF-10 KPIs Financieros:** El sistema debe mostrar en tiempo real el Total de Ingresos, Total de Egresos y el Saldo Neto (Caja actual).
- **RF-11 Gráficos Evolutivos:** Visualización de barras comparativas (Ingresos vs Egresos) de los últimos 6 meses.
- **RF-12 Gráfico de Distribución:** Visualización tipo "Rosquilla" para categorizar los gastos del club.

---

4. Requerimientos No Funcionales (RNF)

RNF-01: Disponibilidad y Rendimiento

- El sistema debe estar disponible 24/7 con un uptime garantizado del 99.9% (SLA VPS ToxiroDigital).
- El tiempo de carga de la interfaz inicial no debe superar los 2 segundos en redes 4G.

RNF-02: Seguridad e Infraestructura

- Toda la comunicación debe estar cifrada bajo protocolo **HTTPS (TLS 1.2/1.3)**.
- El sistema debe desplegarse mediante orquestación de contenedores (**Docker Compose**) para garantizar aislamiento y reproducibilidad.
- La base de datos no debe estar expuesta directamente a internet, solo accesible a través de la red interna de Docker.

RNF-03: Usabilidad y Diseño (UI/UX)

- **Mobile-First:** La interfaz debe adaptarse nativamente a pantallas de smartphones sin pérdida de funcionalidad.
- **Dark Mode:** El esquema de colores debe ser oscuro (#121212) para reducir el consumo de batería y fatiga visual, acorde a la estética "Biker".
- **PWA:** El sistema debe contar con un archivo `manifest.json` y Service Workers básicos para permitir la instalación en la pantalla de inicio del dispositivo.

---

5. Reglas de Negocio

1. **Costo de Licencia:** El uso del sistema y el alojamiento en VPS tiene un costo anual de **$140 USD**, facturado por ToxiroDigital.
2. **Inmutabilidad Histórica:** Las transacciones pasadas no deben ser editables por el socio, solo por el Administrador, para garantizar la integridad contable.

3. **Identidad Única:** No pueden existir dos socios con el mismo correo electrónico.

---

6. Stack Tecnológico

🛠️ Resumen del Stack Tecnológico - Toxiro Digital

Capa	Tecnología	Función Específica
Infraestructura	**VPS Linux (Ubuntu)**	Servidor host dedicado ($140 USD anuales).
Orquestación	**Dokploy / Docker**	Gestión de contenedores y despliegue continuo (CI/CD).
Proxy Inverso	**Traefik**	Terminación SSL (HTTPS) y balanceo de carga.
Base de Datos	**MariaDB / MySQL**	Motor relacional para integridad de datos financieros.
Backend	**Python 3.10+ (FastAPI)**	API asíncrona de alto rendimiento y bajo consumo.
ORM	**SQLAlchemy 2.0**	Mapeo objeto-relacional y seguridad en consultas.
Frontend	**Vanilla JavaScript**	Lógica de cliente pura sin dependencias pesadas.
Diseño / UI	**Bootstrap 5.3 + CSS3**	Interfaz responsiva con estilo "Dark Riders" personalizado.
Seguridad	**SHA-256 (Hashlib)**	Encriptación de credenciales y seguridad de acceso.
Servidor Web	**Uvicorn**	Servidor ASGI para manejo de peticiones concurrentes.

🏗️ Infraestructura y Orquestación

Es el cimiento donde corre todo. Buscamos independencia del hardware mediante la virtualización.

- **Servidor:** VPS de ToxiroDigital ($140 USD anuales) sobre Ubuntu Linux.
- **Orquestador: Dokploy**. Lo utilizamos para gestionar contenedores Docker y facilitar el despliegue automático (CI/CD) cada vez que usted hace un *push* desde su PC.

- **Contenedores: Docker & Docker Compose**. Aislamos el Backend, el Frontend y la Base de Datos para que no dependan de las librerías instaladas en el sistema operativo base.
- **Proxy Inverso: Traefik**. Gestiona el tráfico HTTPS y dirige las peticiones al contenedor correcto.

⚙️ Backend (El Motor)

He elegido un stack moderno que prioriza la velocidad de respuesta y la facilidad de mantenimiento.

- **Lenguaje: Python 3.10+**.
- **Framework API: FastAPI**. Seleccionado por su alto rendimiento (basado en Starlette y Pydantic) y su capacidad de generar documentación automática.
- **ORM: SQLAlchemy 2.0**. Es el traductor entre los objetos de Python y las tablas de la base de datos. Estamos usando la sintaxis moderna para evitar errores de compatibilidad.
- **Servidor ASGI: Uvicorn**. Es el motor que permite a FastAPI manejar múltiples peticiones simultáneas de forma asíncrona.

🗄️ Base de Datos (El Cerebro)

Donde reside la integridad financiera de Dark Riders.

- **Motor: MariaDB / MySQL**. Un estándar de la industria por su robustez y manejo de transacciones.
- **Conector: PyMySQL**. El driver que permite a Python comunicarse con el motor de base de datos.
- **Estrategia:** Relacional, con integridad referencial (Foreign Keys) para asegurar que un movimiento de caja siempre pertenezca a un socio válido.

🎨 Frontend (La Interfaz)

Bajo su requerimiento estricto de minimalismo y limpieza ("estilo GloriaFood"), hemos evitado frameworks pesados como React o Angular.

- **Base: HTML5 y CSS3**. Estructura pura.
- **Diseño: Bootstrap 5.3 (CDN)**. Lo usamos exclusivamente para el sistema de rejilla (Grid) y los componentes base (Modales), pero con personalización de colores para el estilo "Dark Riders".
- **Lógica de Cliente: Vanilla JavaScript (ES6+)**. Sin librerías externas. Todo el manejo de los modales y las llamadas a la API (fetch) es JavaScript puro para garantizar que el sitio cargue instantáneamente.

🛡️ **Seguridad y Otros**

- **Cifrado de Claves: SHA-256 (Hashlib)**. Nunca guardamos contraseñas en texto plano.
- **Manejo de Tiempos: PyTZ**. Para asegurar que todos los registros de tesorería se guarden con la zona horaria de Santiago, Chile.